Agentic Gaussian Splatting: Reinforcement-Learned Control of 3DGS Training for Acceleration

Phase 1 technical description

June 15, 2026

Abstract

We cast the optimization schedule of 3D Gaussian Splatting (3DGS) as a sequential decision problem and train a reinforcement-learning (RL) policy to control it block by block. The policy observes the state of an in-progress reconstruction (loss statistics, held-out validation quality, compute cost, and the distribution of Gaussian primitives) and chooses, once per block of optimizer iterations, how to densify, prune, reset opacity, schedule learning rates, and when to stop. The reward is a time-discounted, model-complexity-aware measure of held-out quality improvement, designed so that maximizing return is equivalent to maximizing early progress on the quality-versus-wall-clock curve—that is, to *accelerating* training. On the held-out `hotdog` scene, the learned policy reaches every test-PSNR target in the 25–33 dB range in 1.24–1.37× less training wall-clock time than the fixed 3DGS schedule, while keeping the model at or below the baseline Gaussian budget.

1 Problem Setting

Stock 3DGS trains a static scene by gradient descent on a photometric loss while periodically *densifying* (splitting and cloning Gaussians in high-gradient regions), *pruning* (removing low-opacity or oversized primitives), and *resetting* opacities. The densification cadence, the gradient and opacity thresholds, the learning-rate schedule, and the stopping iteration are all fixed hyperparameters of a hand-tuned schedule. This schedule is not adaptive: it applies the same recipe regardless of how a particular scene is converging or how the primitive population is evolving.

We replace the fixed schedule with a learned controller. The 3DGS optimizer is left untouched and is wrapped in an environment that exposes its internal state and accepts control actions once per *block* of iterations. A block is a contiguous run of B optimizer steps; the policy acts at block boundaries, so a single episode of T iterations consists of roughly T/B decisions rather than T. This block-wise abstraction keeps the action rate low enough for stable RL while still allowing the controller to adapt within a single training run.

2 Markov Decision Process

We model one training run on one scene as an episode of a Markov decision process $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$. At block k the policy $\pi_\theta(a_k \mid s_k)$ observes state s_k, emits action a_k, the environment executes B_k optimizer iterations under that action, and returns the next state s_{k+1} and a scalar reward r_k. The episode terminates when the iteration budget is exhausted, when the policy chooses to stop, or when a safety limit is hit. All quality signals used for state and reward are computed on a **held-out subset of training cameras**; the dataset test cameras are never read during state

construction or reward computation. They are used only for offline reporting and for checkpoint selection (Section 6).

2.1 State space

The observation is a 45-dimensional vector, every component clipped to $[0, 1]$. It groups into:

Progress (4)
normalized iteration, elapsed wall-clock fraction, remaining-budget fraction, block index.

Loss (4)
exponential moving averages of the L_1 and D-SSIM terms, the total block loss, and the recent loss slope (squashed through tanh).

Validation (4)
held-out PSNR (normalized), SSIM, an LPIPS slot, and the per-block quality improvement.

Compute (5)
time per iteration, time per validation view, peak and allocated VRAM fractions, and the active resolution scale.

Gaussian population, scalar (3)
count fraction, growth rate, and budget utilization.

Previous action (14)
the last block's decoded actions plus a flag for whether they improved validation quality, giving the policy a recurrent handle on its own behavior.

Gaussian population, distributional (11)
opacity quantiles q_{10}, q_{50}, q_{90}, near-transparent fraction, scale quantiles, median anisotropy, visible fraction, added/pruned fractions in the previous block, and the active spherical-harmonic degree fraction.

The distributional statistics are what let the controller reason about *health* of the reconstruction—e.g. a large near-transparent fraction signals prunable clutter, high anisotropy signals degenerate splats—rather than only about scalar loss.

2.2 Action space

Each decision is a hybrid action with six discrete heads (19 categories total) and seven continuous components:

Discrete head	Choices	Effect
`block_steps`	$\{50, 100, 200\}$	length of the next block
`densify_mode`	off / conservative / default / aggressive	scales the gradient threshold
`densification_interval`	$\{50, 100, 200, 400\}$	iterations between densifications
`prune_mode`	off / opacity / opacity+size	pruning criterion
`opacity_reset`	none / if-plateau / force	opacity-reset trigger
`stop`	continue / stop	early termination

Continuous component	Range	Scaling
densify_threshold_mult	$[0.25, 4.0]$	log
prune_opacity_threshold	$[0.001, 0.02]$	linear
position_lr_mult	$[0.25, 4.0]$	log
feature_lr_mult	$[0.25, 2.0]$	log
opacity_lr_mult	$[0.25, 2.0]$	log
scaling_lr_mult	$[0.25, 2.0]$	log
rotation_lr_mult	$[0.25, 2.0]$	log

Continuous outputs are squashed through a sigmoid and mapped onto their range (log- or linear-spaced); a neutral action reproduces the stock 3DGS hyperparameters, so the policy learns *deviations* from a known-good default.

3 Reward

Let Q_k be the held-out validation quality after block k, defined per view as PSNR $+ w_{\text{ssim}}$ SSIM and aggregated across the held-out cameras as

$$Q_k = \bar{q}_k + w_{\min}\,(q_k^{\min} - \bar{q}_k), \tag{1}$$

a blend of the mean and the worst view ($w_{\min} = 0.25$). The worst-view term makes the proxy robust to the policy "trading away" poorly covered directions that the full-hemisphere test cameras would later expose. Let $\Delta Q_k = Q_k - Q_{k-1}$ be the per-block quality gain.

Model-complexity budget. Let N be the current Gaussian count and N_{ref} a per-scene reference budget (set to the initial point-cloud size, $\approx 10^5$). The over-budget excess and a smooth complexity discount are

$$e_k = \max\Big(0,\ \tfrac{N}{N_{\text{ref}}} - 1\Big), \qquad d_k = \frac{1}{1 + \kappa\, e_k}, \qquad \kappa = 2. \tag{2}$$

Time value of progress. The central term for acceleration weights quality gains by the cumulative *training* wall-clock time t_k already spent:

$$\nu_k = \exp\big(-t_k/\tau\big), \qquad \tau = 45\,\text{s}. \tag{3}$$

A plain quality reward telescopes to $Q_{\text{final}} - Q_{\text{initial}}$ and is therefore indifferent to *when* quality is earned; it cannot prefer a faster schedule. Multiplying each gain by ν_k makes early progress worth more, so maximizing return maximizes the early area under the quality-versus-time curve, which is exactly what a time-to-target metric measures. Early stopping emerges endogenously: once the discounted marginal gain falls below the marginal time cost, continuing is no longer rewarded.

Budget-gated growth and compaction. Below budget, growth and pruning are both reward-neutral—the objective is *complexity* $\leq$ *baseline*, not minimal count. Only the over-budget portion of any change is priced:

$$g_k^{+} = \max\big(0,\ N_k - \max(N_{k-1}, N_{\text{ref}})\big) \qquad \text{(growth above budget)}, \tag{4}$$

$$g_k^{-} = \max\big(0,\ N_{k-1} - \max(N_k, N_{\text{ref}})\big) \qquad \text{(reduction above budget)}. \tag{5}$$

Total reward. The block reward combines a discounted quality credit (positive gains discounted by complexity, drops kept at full magnitude), a compute-time penalty, explicit over-budget penalties, a compaction bonus, and stability/resource penalties:

$$r_k = \underbrace{w_q\,\nu_k\big(d_k\,[\Delta Q_k]^+ + [\Delta Q_k]^-\big)}_{\text{time-discounted quality}} - w_t\,\frac{\Delta t_k}{t_{\text{ref}}} \;-\; w_c\,e_k$$
$$-\; w_g\,\frac{g_k^+}{N_{\text{ref}}} \;+\; w_b\,\frac{g_k^-}{N_{\text{ref}}}\,\mathbb{1}[\Delta Q_k \geq -\epsilon] \;-\; w_v\,[\,\text{VRAM}_k - V^\star]^+ \;-\; w_i\,[-\Delta Q_k]^+, \qquad (6)$$

where $[\,\cdot\,]^+$ and $[\,\cdot\,]^-$ denote positive and negative parts. At the terminal block an extra bonus $w_{tq}\,\nu_k\,d_k\,(Q_k - Q_0) - w_{tc}\,e_k$ rewards total quality gain net of final excess, and a numerical-failure penalty guards against divergence. Rewards are standardized per scene (running z-score) before being fed to the RL update, so that scenes with different natural PSNR scales contribute comparably.

4 Safety Rails

The action range is strictly larger than stock 3DGS, which creates failure modes a fixed schedule never encounters. The dominant one is a *prune cliff*: stock 3DGS resets opacities to 0.01 and prunes below 0.005, so survivors recover, but the policy may select a prune threshold above the reset value, wiping the model to the safety floor in a single block. Because validation quality can momentarily *rise* during such a wipe (it removes near-transparent clutter), the reward does not see the damage, and a near-random early policy reliably trips the cliff before learning anything. Two environment-level guards close it—these are constraints on dynamics, not incentives, and reward tuning alone cannot replace them:

1. **Post-reset prune guard.** For a recovery window of 300 iterations after any opacity reset, the effective prune threshold is capped at the stock value 0.005.
2. **Per-event prune cap.** A single densify/prune event may remove at most 25% of the current population, so reaching the floor requires sustained pruning across blocks that the reward can observe and penalize.

Additional rails bound opacity-reset frequency, an absolute Gaussian floor, peak VRAM, and a minimum iteration count before the `stop` action is honored.

5 Policy and Optimization

Architecture. A shared multilayer perceptron (3 hidden layers of width 256, GELU activations) encodes the state. From the shared features, six independent categorical heads produce the discrete-action logits, a diagonal Gaussian head (learned log-std) produces the seven continuous actions, and a scalar critic head produces the value estimate.

Training. The policy is optimized with Proximal Policy Optimization (PPO): clipped surrogate objective (clip 0.2), generalized advantage estimation ($\gamma = 0.99$, $\lambda = 0.95$), advantage normalization, value clipping, gradient-norm clipping, an entropy bonus, and KL-based early stopping of the inner epochs. Rollouts span multiple episodes across a diverse set of training scenes (`chair`, `drums`, `mic`, `ficus`, `ship`); scene diversity is essential, since a controller trained only on easy scenes

overfits to their capacity profile and transfers poorly. The hybrid log-probability is the sum of the categorical log-probabilities and the Gaussian log-probabilities, and the entropy bonus sums both, so exploration is driven jointly over discrete and continuous decisions.

6 Acceleration-Aware Checkpoint Selection

Because the train-camera reward is a proxy, over-optimizing it can widen the train-/test-camera gap; selecting checkpoints by reward or by update count reproduced a regression in which *more* training yielded *worse* test PSNR. We therefore select checkpoints with a held-out *probe*: every few PPO updates, the current policy runs one frozen, deterministic episode on a held-out probe scene (neither a training nor an evaluation scene), scored on that scene's **test** cameras. Crucially, the probe is scored for *acceleration*: test PSNR is sampled on a fast view subset whenever cumulative training wall-clock time crosses $\{15, 30, 60\}$ s, and the selection metric is the mean of these time-sliced PSNRs—an approximation to the early area under the quality-time curve. The checkpoint maximizing this score is saved as the deployed policy. With this metric the objective improves monotonically over training, the opposite of the reward-/count-selected regression.

7 Results

We evaluate with a time-to-target protocol: both the fixed 3DGS schedule and the frozen policy are run under matched iteration budgets and the same camera split, test PSNR is measured on the full test set, and the training wall-clock time to reach each target PSNR is read off the PSNR-time Pareto envelope (linearly interpolated between samples). On the held-out `hotdog` scene:

Target PSNR (dB)	**Baseline (s)**	**Agentic (s)**	**Speedup**
25	9.72	7.08	1.37×
27	14.17	11.34	1.25×
29	18.27	13.91	1.31×
31	22.37	17.99	1.24×
33	30.98	24.22	1.28×

The learned controller dominates the fixed schedule across the target range at a constant, baseline-sized Gaussian budget, reaching each quality level in 1.24–1.37× less training time. The policy stops itself at roughly 3000 iterations (a self-discovered consequence of the time-discounted reward), which caps its frontier near 34 dB; pushing the high-PSNR range further is a matter of lengthening the time-value horizon τ. Figure 1 shows the full quality-versus-time curves.

8 Summary

The contribution is a control reformulation of 3DGS training in which an RL policy adaptively schedules densification, pruning, opacity resets, learning rates, and stopping, driven by a reward whose maximizer is fast early convergence at a bounded model size. Three design elements make it work: budget-gated complexity terms that make "complexity $\leq$ baseline" the operative constraint without forcing minimal count; dynamics-level safety rails that remove an otherwise unavoidable model-wipe failure mode; and an acceleration-aware probe that selects checkpoints by time-to-quality on held-out test cameras rather than by training reward. Together they yield a controller that accelerates held-out reconstruction by 1.24–1.37× at matched quality and budget.

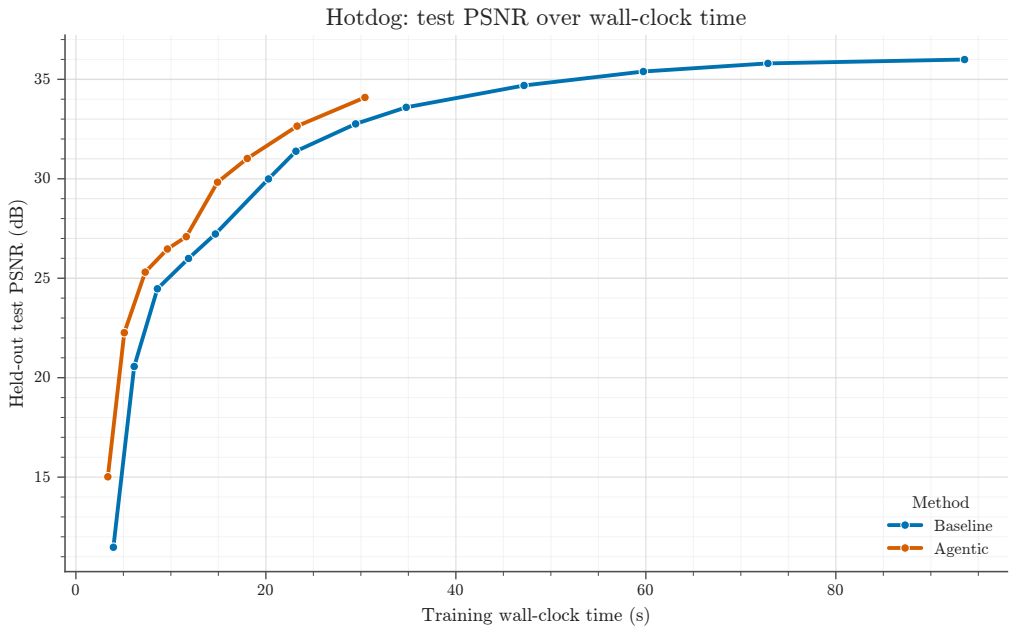

Figure 1: Held-out test PSNR versus training wall-clock time on `hotdog`. The agentic policy (orange) reaches every quality level to the left of the fixed 3DGS schedule (blue), then stops itself near 34 dB; the baseline continues to a higher ceiling at greater cost.